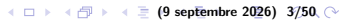


Cours MVSI
Modélisation et Vérification
des Systèmes Informatiques

Modélisation, spécification et vérification (I)

Dominique Méry
Telecom Nancy, Université de Lorraine
(9 septembre 2026 at 1:31 P.M.)

- ① Programming by Contract
- ② Summary of the Tryptich
- ③ Transition Systems
 - Overview of Transition Systems as Modelling Tool
 - Expression of transition systems
 - Main concepts of discrete transition system
 - Expression of discrete transition systems
- ④ Transition system in action with TLA/TLA⁺
 - GCD
 - Simple Access Control
 - TLA / TLA⁺



Method for verifying program properties

A program P satisfies a (pre,post) contract :

- ▶ P transforms a variable x from initial values x_0 and produces a final value $x_f : x_0 \xrightarrow{P} x_f$
- ▶ x_0 satisfies pre : $\text{pre}(x_0)$ and x_f satisfies post : $\text{post}(x_0, x_f)$
- ▶ $\text{pre}(x_0) \wedge x_0 \xrightarrow{P} x_f \Rightarrow \text{post}(x_0, x_f)$
- ▶ $\mathbb{D}$ est le domaine RTE de X

```
requires  $\text{pre}(x_0)$   
ensures  $\text{post}(x_0, x_f)$   
variables  
 $\text{mathhtt{tx}}$ 
```

```
begin  
  0 :  $P_0(x_0, x)$   
  instruction0  
  ...  
  i :  $P_i(x_0, x)$   
  ...  
  instructionf-1  
  f :  $P_f(x_0, x)$   
end
```

- ▶ $\text{pre}(x_0) \wedge x = x_0 \Rightarrow P_0(x_0, x)$
- ▶ $\text{pre}(x_0) \wedge P_f(x_0, x) \Rightarrow \text{post}(x_0, x)$
- ▶ For any pair of labels ℓ, ℓ' such that $\ell \longrightarrow \ell'$, one verifies that, for any values $x, x' \in \text{MEMORY}$
$$\left(\begin{array}{l} \text{pre}(x_0) \wedge P_\ell(x_0, x) \\ \wedge \text{cond}_{\ell, \ell'}(x) \wedge x' = f_{\ell, \ell'}(x) \end{array} \right) \Rightarrow P_{\ell'}(x_0, x')$$
- ▶ For any pair of labels m, n such that $m \longrightarrow n$, one verifies that,
 $\forall x, x' \in \text{MEMORY} :$
 $\text{pre}(x_0) \wedge P_m(x_0, x) \Rightarrow \mathbf{DOM}(m, n)(x)$

Example of an annotation

```
VARIABLES  $X$   
REQUIRES ...  
ENSURES ...  
WHILE  $0 < X$  DO  
   $X := X - 1$ ;  
OD;
```

Example of an annotation

```
VARIABLES  $X$   
REQUIRES ...  
ENSURES ...  
WHILE  $0 < X$  DO  
   $X := X - 1$ ;  
OD;
```



Example of an annotation

```
VARIABLES  $X$   
REQUIRES ...  
ENSURES ...  
WHILE  $0 < X$  DO  
   $X := X - 1$ ;  
OD;
```

→ →

Example of an annotation

```
VARIABLES  $X$ 
REQUIRES ...
ENSURES ...
WHILE  $0 < X$  DO
   $X := X - 1$ ;
OD;
```



```

CONTRACT  $EX$ 
VARIABLES  $X(int)$ 
REQUIRES  $x_0 \in \mathbb{N}$ 
ENSURES  $x_f = 0$ 
   $\ell_0 : \{ x = x_0 \wedge x_0 \in \mathbb{N} \}$ 
WHILE  $0 < X$  DO
   $\ell_1 : \{ 0 < x \leq x_0 \wedge x_0 \in \mathbb{N} \}$ 
     $X := X - 1;$ 
   $\ell_2 : \{ 0 \leq x \leq x_0 \wedge x_0 \in \mathbb{N} \}$ 
OD;
   $\ell_3 : \{ x = 0 \}$ 

```


- ▶ $\mathcal{R}$: system requirements.

- ▶ $\mathcal{R}$: system requirements.
- ▶ $\mathcal{D}$: problem domain.

- ▶ $\mathcal{R}$: system requirements.
- ▶ $\mathcal{D}$: problem domain.
- ▶ $\mathcal{S}$: system specification.

- ▶ $\mathcal{R}$: system requirements.
- ▶ $\mathcal{D}$: problem domain.
- ▶ $\mathcal{S}$: system specification.

$\mathcal{D}, \mathcal{S}$ SATISFIES $\mathcal{R}$

- ▶ $\mathcal{R}$: system requirements.
- ▶ $\mathcal{D}$: problem domain.
- ▶ $\mathcal{S}$: system specification.

$\mathcal{D}, \mathcal{S}$ SATISFIES $\mathcal{R}$

- ▶ $\mathcal{R}$: pre/post.
- ▶ $\mathcal{D}$: integers, reals, ...
- ▶ $\mathcal{S}$: code, procedure, program, ...

Method for verifying program properties correctness(PC,RTE)

A program P satisfies a (pre,post) contract :

- ▶ P transforms a variable v from initial values v_0 and produces a final value $v_f : v_0 \xrightarrow{P} v_f$
- ▶ v_0 satisfies pre : $\text{pre}(v_0)$ and v_f satisfies post : $\text{post}(v_0, v_f)$
- ▶ $\text{pre}(v_0) \wedge v_0 \xrightarrow{P} v_f \Rightarrow \text{post}(v_0, v_f)$
- ▶ $\mathbb{D}$ est le domaine RTE de V

requires $\text{pre}(v_0)$

ensures $\text{post}(v_0, v_f)$

variables X

begin

$0 : P_0(v_0, v)$

instruction₀

...

$i : P_i(v_0, v)$

...

instruction _{$f-1$}

$f : P_f(v_0, v)$

end

▶ $\text{pre}(v_0) \wedge v = v_0 \Rightarrow P_0(v_0, v)$

▶ $\text{pre}(v_0) \wedge P_f(v_0, v) \Rightarrow \text{post}(v_0, v)$

▶ For any pair of labels ℓ, ℓ'
such that $\ell \longrightarrow \ell'$, one verifies that,
pour any values $v, v' \in \text{MEMORY}$
$$\left(\begin{array}{l} \text{pre}(v_0) \wedge P_\ell(v_0, v) \\ \wedge \text{cond}_{\ell, \ell'}(v) \wedge v' = f_{\ell, \ell'}(v) \end{array} \right) \Rightarrow P_{\ell'}(v_0, v')$$

▶ For any pair of labels m, n
such taht $m \longrightarrow n$, one verifies that,
 $\forall v, v' \in \text{MEMORY} :$

$\text{pre}(v_0) \wedge P_m(v_0, v) \Rightarrow \text{DOM}(m, n)(v)$

Example of an annotation

```
VARIABLES  $X$   
REQUIRES ...  
ENSURES ...  
WHILE  $0 < X$  DO  
   $X := X - 1$ ;  
OD;
```


Example of an annotation

```
VARIABLES  $X$   
REQUIRES ...  
ENSURES ...  
WHILE  $0 < X$  DO  
   $X := X - 1;$   
OD;
```



Example of an annotation

```
VARIABLES  $X$   
REQUIRES ...  
ENSURES ...  
WHILE  $0 < X$  DO  
   $X := X - 1;$   
OD;
```

→ →

$\longrightarrow \longrightarrow \longrightarrow$

Example of an annotation

```
VARIABLES  $X$   
REQUIRES ...  
ENSURES ...  
WHILE  $0 < X$  DO  
   $X := X - 1$ ;  
OD;
```

→ → → →

Example of an annotation

VARIABLES X
REQUIRES ...
ENSURES ...
WHILE $0 < X$ **DO**
 $X := X - 1;$
OD;

→ → → →

CONTRACT EX
VARIABLES $X(int)$
REQUIRES $x_0 \in \mathbb{N}$
ENSURES $x_f = 0$
 $\ell_0 : \{ x = x_0 \wedge x_0 \in \mathbb{N} \}$
WHILE $0 < X$ **DO**
 $\ell_1 : \{ 0 < x \leq x_0 \wedge x_0 \in \mathbb{N} \}$
 $X := X - 1;$
 $\ell_2 : \{ 0 \leq x \leq x_0 \wedge x_0 \in \mathbb{N} \}$
OD;
 $\ell_3 : \{ x = 0 \}$

Transition system

A transition system $\mathcal{ST}$ is given by a set of states Σ , a set of initial states $Init$ and a binary relation $\mathcal{R}$ on Σ .

- ▶ The set of terminal states $Term$ defines specific states, identifying particular states associated with a termination state and this set can be empty, in which case the transition system does not terminate.

event

A transformation is caused by an event that updates a temperature from a sensor, or a computer updating a computer variable, or an actuator sending a signal to a controlled entity.

An observation of a system S is based on the following points :

- ▶ a state $s \in \Sigma$ allows you to observe elements and reports on these elements, such as the number of people in the meeting room or the capacity of the room : $s(np)$ and $s(cap)$ are two positive integers.
- ▶ a relationship between two states s and s' observes a transformation of the state s into a state s' and we will note $s \xrightarrow{R} s'$ which expresses the observation of a relationship R :
 $R = s(np) \in 0..s(cap)-1 \wedge s'(np) = s(np)+1 \wedge s'(cap) = s(cap)$ is an expression of R observing that one more person has entered the room.
- ▶ a trace $s_0 \xrightarrow{R_0} s_1 \xrightarrow{R_1} s_2 \xrightarrow{R_2} s_3 \xrightarrow{R} \dots \xrightarrow{R_{i-1}} s_i \xrightarrow{R_i} \dots$ is a trace generated by the different observations $R_0, \dots R_p, \dots$

- ▶ observing changes of state that correspond either to physical or biological phenomena or to artefactual structures such as a program, a service or a platform.
- ▶ An observation generally leads to the identification of a few possible transformations of the observed state, and the closed-model hypothesis follows naturally.
- ▶ One consequence is that there are visible transformations and invisible transformations.
- ▶ These invisible transformations of the state are expressed by an identity relation called event skip (or stuttering [?]).
- ▶ A modelling produces a closed model with a skip event modelling what is not visible in the observed state.

- ▶ hypothesis : a system S is modelled by a set of states Σ , and $\Sigma \stackrel{def}{=} \text{Var} \longrightarrow D$ where Var is the variable (or list of variables) of the system S and D is the domain of possible values of variables.
- ▶ The interpretation of a formula P in a state $s \in \Sigma$ is denoted $\llbracket P \rrbracket(s)$ or sometimes $s \in \hat{P}$.
- ▶ A distinction is made between flexible variable symbols x and logical variable symbols v , and constant symbols c are used.

- ① $\llbracket \mathbf{x} \rrbracket(s) = s(\mathbf{x}) = x : x$ is the value of the variable $\mathbf{x}$ in s .
- ② $\llbracket \mathbf{x} \rrbracket(s') = s'(\mathbf{x}) = x' : x'$ is the value of the variable $\mathbf{x}$ in s' .
- ③ $\llbracket c \rrbracket(s)$ is the value of c in s , in other words the value of the constant c in s .
- ④ $\llbracket \varphi(x) \wedge \psi(x) \rrbracket(s) = \llbracket \varphi(x) \rrbracket(s)$ et $\llbracket \psi(x) \rrbracket(s)$ where *and* is the classical interpretation of symbol $\wedge$ according to the truth table.
- ⑤ $\llbracket \mathbf{x} = 6 \wedge y = \mathbf{x} + 8 \rrbracket(s) \stackrel{def}{=} \llbracket \mathbf{x} \rrbracket(s) = \llbracket 6 \rrbracket(s)$ **and** $\llbracket y \rrbracket(s) = \llbracket x \rrbracket(s) + \llbracket 8 \rrbracket(s) = (x = 6$ **and** $y = x + 8$ where y is a logical variable distinct of $\mathbf{x}$ and where $\llbracket \mathbf{x} \rrbracket(s) = s(\mathbf{x}) = x$.

- ▶ $\llbracket x \rrbracket(s)$ is the value of x in s and its value will be distinguished by the font used : x is the `tt` font of $\text{\LaTeX}$ and x is the math font of $\text{\LaTeX}$.
- ▶ Using the name of the variable x as its current value, i.e. x and $\llbracket x \rrbracket(s')$ is the value of x in s' and will be noted x' .
- ▶ The transition relation as a relation linking the state of the variables in s and the state of the variables in s' using the prime notation as defined by L. Lamport for TLA.
- ▶ Types of variable depending on whether we are talking about the computer variable, its value or whether we are defining constants such as np , the number of processes, or π , which designates the constant π .
- ▶ a current observation refers to a current state for both enduring and perdurant information data in the sense of the Dines Bjørner.

flexible variable

A flexible variable x is a name related to a perdurant information according to a state of the (current observed) system :

- ▶ x is the current value of x in other words the value at the observation time of x .
- ▶ x' is the next value of x in other words the value at the next observation time of x .
- ▶ x_0 is the initial value of x in other words the value at the initial observation time of x .

A logical variable x is a name related to an endurant entity designated by this name.

state property of a system

Let be a system S whose flexible variables x are the elements of $\mathcal{Var}(S)$. A property $P(x)$ of S is a logical expression involving ,freely the flexible variables x and whose interpretation is the set of values of the domain of x : $P(x)$ is true in x , if the value x satisfies $P(x)$.

For each property $P(x)$, we can associate a subset of D denoted $\hat{P}$ and, in this case, $P(x)$ is true in x . is equivalent to $x \in \hat{P}$.

Examples of property

- ▶ $P_1(x) \stackrel{def}{=} x \in 18..22 : x$ is a value between 18 and 22 and $\hat{P}_1 = \{18, 19, 20, 21, 22\}$.
- ▶ $P_2(p) \stackrel{def}{=} p \subset PEOPLE \wedge card(p) \leq n : p$ is a set of persons and that set has at most n elements and $\hat{P}_2 = \{p_1 \dots p_n\}$. In this example, we use a logical variable n and a name for a constant $PEOPLE$.

basic set of a system S

The list of symbols $s_1, s_2, \dots, s_p$ corresponds to the list of basic set symbols in the D domain of S and $s_1 \cup \dots \cup s_p \subseteq D$.

constants of system S

The list of symbols $c_1, c_2, \dots, c_q$ corresponds to the list of symbols for the constants of S .

Examples of constant and set

- ▶ *fred* is a constant and is linked to the set *PEOPLE* using the expression $fred \in PEOPLE$ which means that *fred* is a person from *PEOPLE*.
- ▶ *aut* is a constant which is used to express the table of authorisations associated with the use of vehicles. the expression $aut \subseteq PEOPLE \times CARS$ where *CARS* denotes a set of cars.

.....

⊠ Definition(axiomatics for S)

The list of axioms of S is called the axiomatics of S and is denoted $AX(S, s, c)$ where s denotes the basic sets and c denotes the constants of S.

.....

.....

⊠ Definition(theorem for S)

A property $P(s, c)$ is a theorem for S, if $AX(S, s, c) \vdash P(s, c)$ is a valid sequent.

Theorems for S are denoted by $TH(S, s, c)$.

.....

Let s, s' be two states of S ($s, s' \in \mathcal{Var}(S) \longrightarrow \mathbf{VALS}$). $s \xrightarrow{R} s'$ will be written as a relation $R(x, x')$ where x and x' designate values of $\mathbf{x}$ before and after the observation of R.

.....

⊠ Definition(event)

Let $\mathcal{Var}(S)$ be the set of flexible variables of S. Let s be the basis sets and c the constants of S. An event e for S is a relational expression of the form $R(s, c, x, x')$ denoted $RA(e)(s, c, x, x')$.

.....

⊠ Definition(event-based model of a system)

Let $\mathcal{Var}(S)$ be the set of flexible variables of S denoted x . Let s be the list of basis sets of the system S . Let c be the list of constants of the system S . Let D be a domain containing sets s . An event-based model for a system S is defined by

$$(AX(s, c), x, \text{VALS}, \text{Init}(x), \{e_0, \dots, e_n\})$$

where

- ▶ $AX(s, c)$ is an axiomatic theory defining the sets, constants and static properties of these elements.
- ▶ $\text{Init}(x)$ defines the possible initial values of x .
- ▶ $\{e_0, \dots, e_n\}$ is a finite set of events of S and e_0 is a particular event present in each event-based model defined by
 $BA(e_0)(x, x') = (x' = x)$.

The event-based model is denoted

$$EM(s, c, x, \text{VALS}, \text{Init}(x) \{e_0, \dots, e_n\}) = (AX(s, c), x, \text{VALS}, \text{Init}(x), \{e_0, \dots, e_n\}).$$

- ▶ $Next(x, x')$ or
 $Next(s, c, x, x') \stackrel{def}{=} BA(e_0)(s, c, x, x') \vee \dots \vee BA(e_n)(s, c, x, x')$.
- ▶ the transitive reflexive closure of the relation $Next^*(s, c, x_0, x) \stackrel{def}{=} \begin{cases} \vee x = x_0 \\ \vee Next(s, c, x_0, x) \\ \vee \exists xi \in VALS. Next^*(s, c, x_0, xi) \wedge Next(s, c, xi, x) \end{cases}$

.....
☒ Definition(safety property)

A property $P(x)$ is a safety property for the system S , if

$$\forall x_0, x \in VALS. Init(x_0) \wedge Next^*(s, c, x_0, x) \Rightarrow P(x).$$

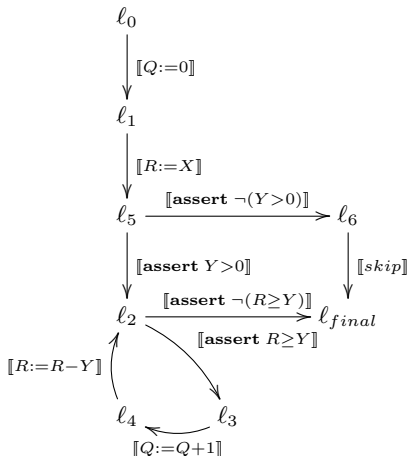
.....

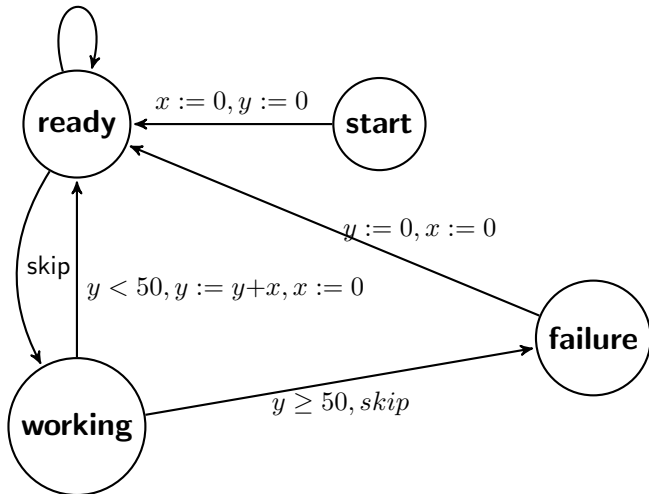
Programme as flowchart

```

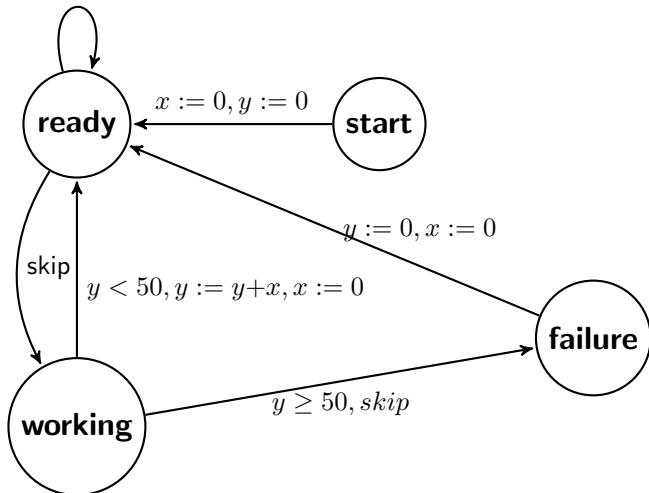
 $\ell_0[Q := 0];$ 
 $\ell_1[R := X];$ 
IF  $\ell_5[Y > 0]$ 
    WHILE  $\ell_2[R \geq Y]$ 
         $\ell_3[Q := Q+1];$ 
         $\ell_4[R := R-Y]$ 
    ENDWHILE
ELSE
     $\ell_6[skip]$ 
ENDIF

```



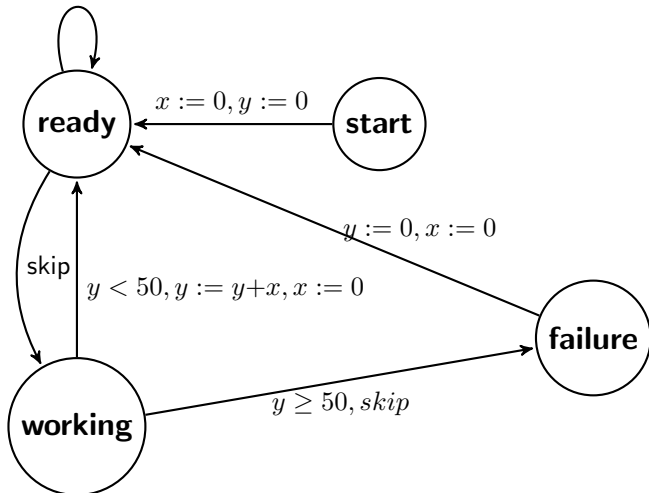
$$x \leq 5, x := x+1$$


A small system as an automaton

$$x \leq 5, x := x+1$$


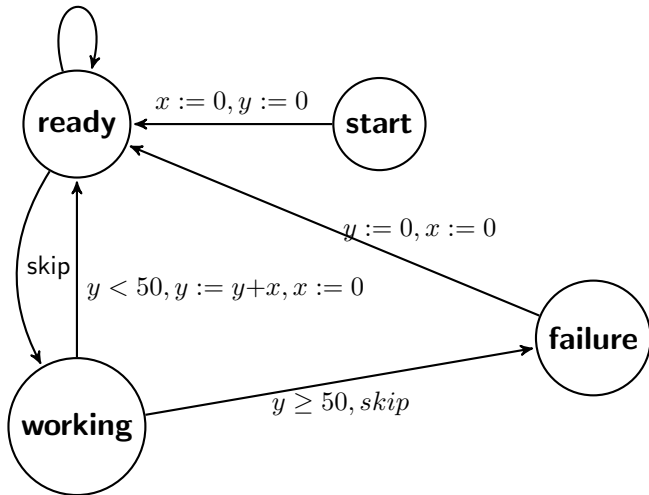
► safety1 : $0 \leq x \leq 5$

A small system as an automaton

$$x \leq 5, x := x+1$$


► safety1 : $0 \leq x \leq 5$ et ...

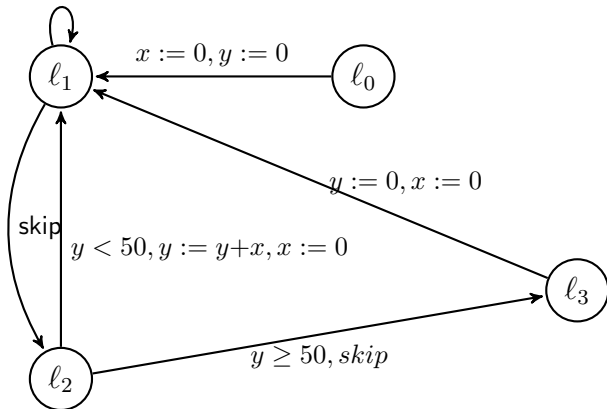
A small system as an automaton

$$x \leq 5, x := x+1$$


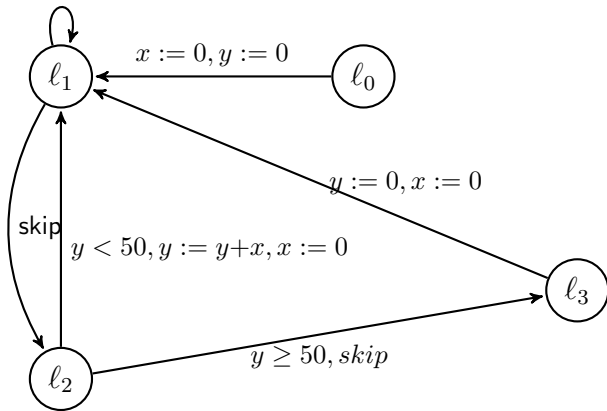
► safety1 : $0 \leq x \leq 5$ et ... safety2 : $0 \leq y \leq 56$

A small system as an automaton

$x \leq 5, x := x+1$

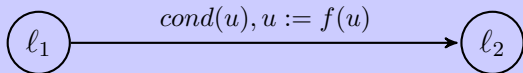


A small system as an automaton

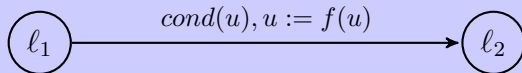
$$x \leq 5, x := x+1$$


- ▶ $\text{safety1} : 0 \leq x \leq 5$ et $\text{safety2} : 0 \leq y \leq 56$
- ▶ $\text{skip} = x := x, y := y$
- ▶ $\text{skip} = \text{TRUE}, x := x, y := y = \text{TRUE}, \text{skip}$

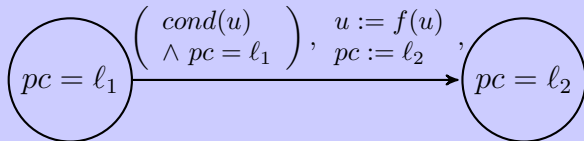
Transition between two control states

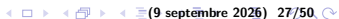


Transition between two control states



Transition between two control states





Un modèle relationnel $\mathcal{MS}$ pour un système $\mathcal{S}$ est une structure

$$(Th(s, c), X, VALS, INIT(x), \{r_0, \dots, r_n\})$$

où

- ▶ $Th(s, c)$ est une théorie définissant les ensembles, les constantes et les propriétés statiques de ces éléments.
- ▶ X est une liste de variables flexibles.
- ▶ $VALS$ est un ensemble de valeurs possibles pour X .
- ▶ $\{r_0, \dots, r_n\}$ est un ensemble fini de relations reliant les valeurs avant x et les valeurs après x' .
- ▶ $INIT(x)$ définit l'ensemble des valeurs initiales de X .
- ▶ la relation r_0 est la relation $Id[VALS]$, identité sur $VALS$.

.....

☒ Definition

Soit $(Th(s, c), X, VALS, INIT(x), \{r_0, \dots, r_n\})$ un modèle relationnel d'un système $\mathcal{S}$. La relation NEXT associée à ce modèle est définie par la disjonction des relations r_i :

$$NEXT \stackrel{def}{=} r_0 \vee \dots \vee r_n$$

.....

pour une variable x , nous définissons les valeurs suivantes :

- ▶ x est la valeur courante de la variable X.
- ▶ x' est la valeur suivante de la variable X.
- ▶ x_0 ou $\underline{x}$ sont la valeur initiale de la variable X.
- ▶ $\bar{x}$ ou x_f est la valeur finale de la variable X, quand cette notion a du sens.

.....

⊠ Definition(assertion)

Soit $(Th(s, c), X, VALS, INIT(x), \{r_0, \dots, r_n\})$ un modèle relationnel M d'un système $\mathcal{S}$. Une propriété A est une propriété assertionnelle de sûreté pour le système $\mathcal{S}$, si

$$\forall x_0, x \in VALS. Init(x_0) \wedge NEXT^*(x_0, x) \Rightarrow A(x).$$

.....

.....

⊠ Definition(relation)

Soit $(Th(s, c), X, VALS, INIT(x), \{r_0, \dots, r_n\})$ un modèle relationnel M d'un système $\mathcal{S}$. Une propriété R est une propriété relationnelle de sûreté pour le système $\mathcal{S}$, si

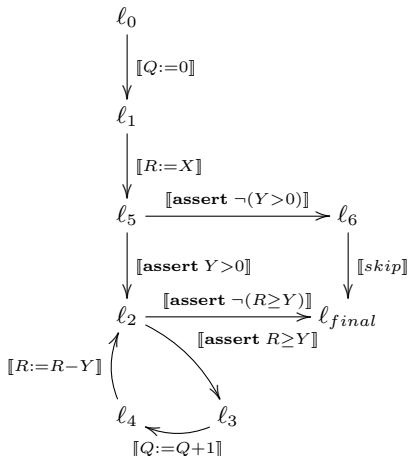
$$\forall x_0, x \in VALS. Init(x_0) \wedge NEXT^*(x_0, x) \Rightarrow R(x_0, x).$$

.....

- ▶ P. et R. Cousot développent une étude complète des propriétés d'invariance et de sûreté en mettant en évidence correspondances entre les différentes méthodes ou systèmes proposées par Turing, Floyd, Hoare, Wegbreit, Manna ... et reformulent les principes d'induction utilisés pour définir ces méthodes de preuve (voir les deux cubes des 16 principes).

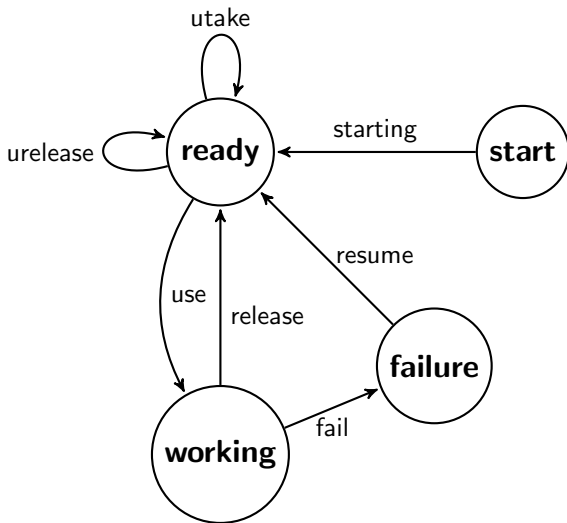


```
ℓ0[Q := 0];  
ℓ1[R := X];  
IF ℓ5[Y > 0]  
    WHILE ℓ2[R ≥ Y]  
        ℓ3[Q := Q + 1];  
        ℓ4[R := R - Y]  
    ENDWHILE  
ELSE  
    ℓ6[skip]  
ENDIF
```

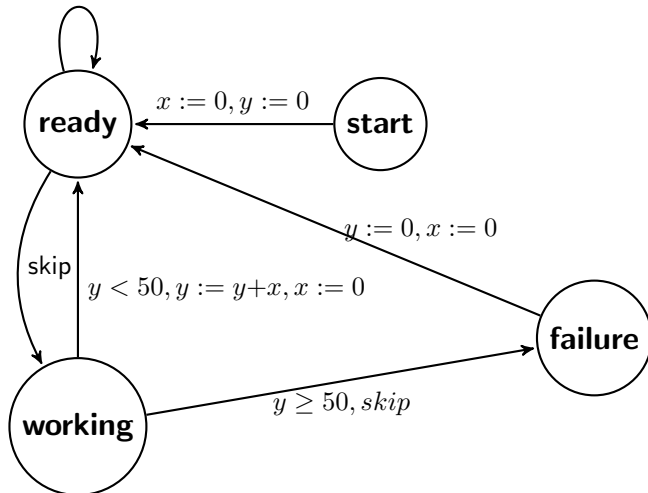


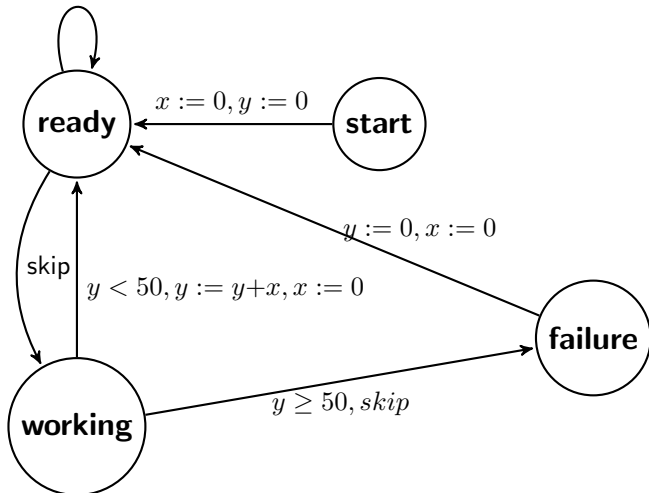
- ▶ Un automate a des états de contrôle : compteur ordinal d'un programme
- ▶ Un automate a des étiquettes : événements, actions, ...
- ▶ Un automate peut aussi avoir des variables explicites qui sont modifiées par des actions
- ▶ Un automate décrit des exécutions possibles qui sont des chemins suivant les informations de l'automate.

Un petit système en tant qu'automate

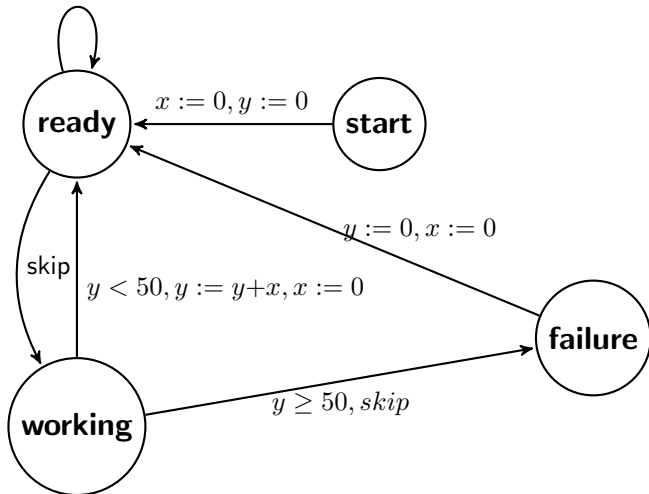


$x \leq 5, x := x+1$

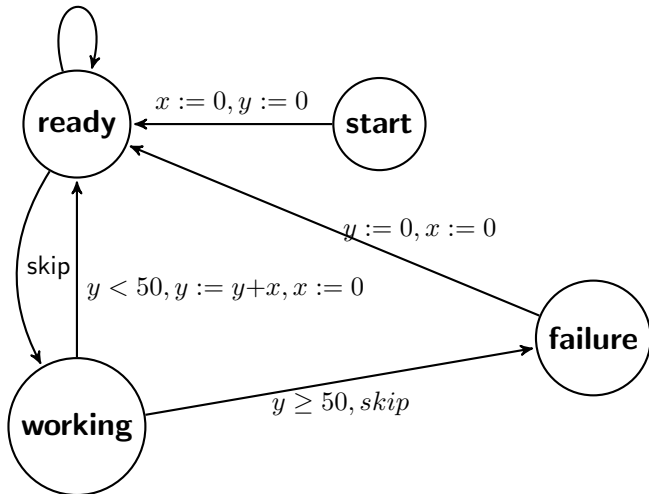


$$x \leq 5, x := x+1$$


► safety1 : $0 \leq x \leq 5$

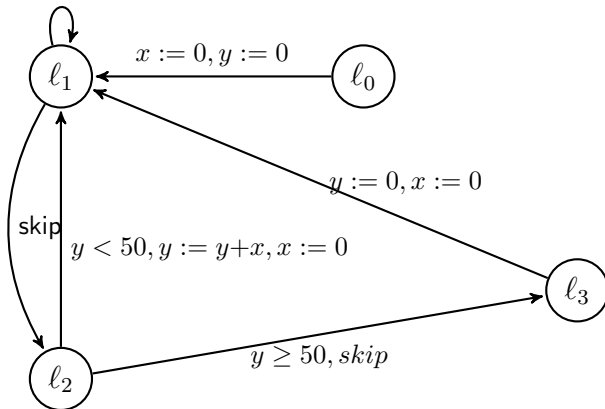
$$x \leq 5, x := x+1$$


► safety1 : $0 \leq x \leq 5$ et ...

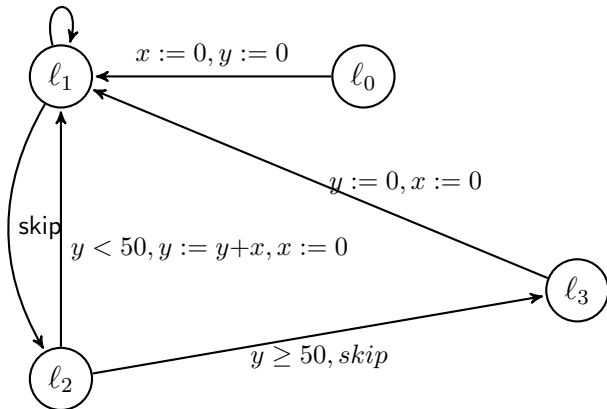
$$x \leq 5, x := x+1$$


► safety1 : $0 \leq x \leq 5$ et ... safety2 : $0 \leq y \leq 56$

$x \leq 5, x := x+1$

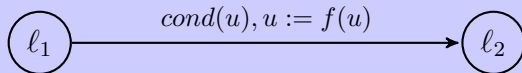


Un petit système en tant qu'automate

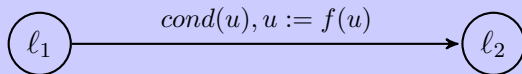
$$x \leq 5, x := x+1$$


- ▶ $\text{safety1} : 0 \leq x \leq 5$ et $\text{safety2} : 0 \leq y \leq 56$
- ▶ $\text{skip} = x := x, y := y$
- ▶ $\text{skip} = \text{TRUE}, x := x, y := y = \text{TRUE}, \text{skip}$

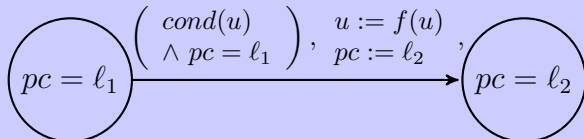
Transition entre deux états de contrôle



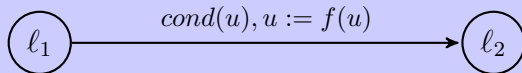
Transition entre deux états de contrôle



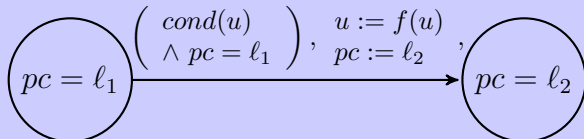
Transition entre deux états de contrôle



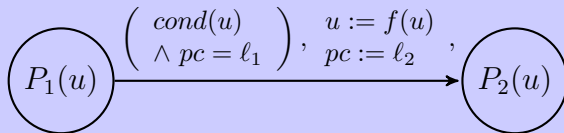
Transition entre deux états de contrôle



Transition entre deux états de contrôle



Transition entre deux prédicats



MODULE *pgcd*

EXTENDS *Naturals, TLC*

CONSTANTS *a, b*

VARIABLES *x, y*

Init $\triangleq x = a \wedge y = b$

a1 $\triangleq x > y \wedge x' = x - y \wedge y' = y$

a2 $\triangleq x < y \wedge y' = y - x \wedge x' = x$

over $\triangleq x = y \wedge x' = x \wedge y' = y$

Next $\triangleq a_1 \vee a_2 \vee over$

test $\triangleq x \neq y$

MODULE *ex1*

modules de base importables

EXTENDS *Naturals*, *TLC*

un système contrôle l'accès à une salle dont la capacité est de 19 personnes ; écrire
un modèle de ce système en vérifiant la propriété de sûreté

VARIABLES *np*

Première tentative

$$\text{entrer} \triangleq np' = np + 1$$
$$\text{sortir} \triangleq np' = np - 1$$
$$\text{next} \triangleq \text{entrer} \vee \text{sortir}$$
$$\text{init} \triangleq np = 0$$


Seconde tentative

$$\text{entrer}_2 \triangleq np < 19 \wedge np' = np + 1$$
$$\text{next}_2 \triangleq \text{entrer}_2 \vee \text{sortir}$$

Troisième tentative

$$\text{sortir}_2 \triangleq np > 0 \wedge np' = np - 1$$

$$\text{next}_3 \triangleq \text{entrer}_2 \vee \text{sortir}_2$$

$$\text{safety}_1 \triangleq np \leq 19$$

$$\text{question}_1 \triangleq np \neq 6$$

Soit $(Th(s, c), x, VALS, INIT(x), \{r_0, \dots, r_n\})$ un modèle relationnel M d'un système $\mathcal{S}$. Une propriété A est une propriété de sûreté pour le système $\mathcal{S}$, si

$$\forall x_0, x \in VALS. Init(x_0) \wedge NEXT^*(x_0, x) \Rightarrow A(x).$$

Soit $(Th(s, c), x, VALS, INIT(x), \{r_0, \dots, r_n\})$ un modèle relationnel M d'un système $\mathcal{S}$. Une propriété A est une propriété de sûreté pour le système $\mathcal{S}$, si

$\forall x_0, x \in VALS. Init(x_0) \wedge NEXT^*(x_0, x) \Rightarrow A(x).$

► x est une variable ou une liste de variable : VARIABLES x

Soit $(Th(s, c), x, VALS, INIT(x), \{r_0, \dots, r_n\})$ un modèle relationnel M d'un système $\mathcal{S}$. Une propriété A est une propriété de sûreté pour le système $\mathcal{S}$, si

$\forall x_0, x \in VALS. Init(x_0) \wedge NEXT^*(x_0, x) \Rightarrow A(x).$

- ▶ x est une variable ou une liste de variable : `VARIABLES x`
- ▶ $Init(x)$ est une variable ou une liste de variable : `init == Init(x)`

Soit $(Th(s, c), x, VALS, INIT(x), \{r_0, \dots, r_n\})$ un modèle relationnel M d'un système $\mathcal{S}$. Une propriété A est une propriété de sûreté pour le système $\mathcal{S}$, si

$$\forall x_0, x \in VALS. Init(x_0) \wedge NEXT^*(x_0, x) \Rightarrow A(x).$$

- ▶ x est une variable ou une liste de variable : `VARIABLES x`
- ▶ $Init(x)$ est une variable ou une liste de variable : `init == Init(x)`
- ▶ $NEXT^*(x_0, x)$ est la définition de la relation définissant ce que fait le système : `Next == a1 \/\ a2 \/\ .... \/\ an`

Soit $(Th(s, c), x, VALS, INIT(x), \{r_0, \dots, r_n\})$ un modèle relationnel M d'un système $\mathcal{S}$. Une propriété A est une propriété de sûreté pour le système $\mathcal{S}$, si

$$\forall x_0, x \in VALS. Init(x_0) \wedge NEXT^*(x_0, x) \Rightarrow A(x).$$

- ▶ x est une variable ou une liste de variable : `VARIABLES x`
- ▶ $Init(x)$ est une variable ou une liste de variable : `init == Init(x)`
- ▶ $NEXT^*(x_0, x)$ est la définition de la relation définissant ce que fait le système : `Next == a1 \/\ a2 \/\ .... \/\ an`
- ▶ $A(x)$ est une expression logique définissant une propriété de sûreté à vérifier sur toutes les configurations du modèle : `Safety == A(x)`

- ▶ TLA (Temporal Logic of Actions) sert à exprimer des formules en logique temporelle : $\Box P$ ou *toujours P*
- ▶ TLA⁺ est un langage permettant de déclarer des constantes, des variables et des définitions :
 - $\langle \text{def} \rangle == \langle \text{expression} \rangle$: une définition $\langle \text{def} \rangle$ est la donnée d'une expression $\langle \text{expression} \rangle$ qui utilise des éléments définis avant ou dans des modules qui *étendent* ce module.
 - Une variable x est soit sous la forme x soit sous la forme x' : x' est la valeur de x après.
 - Un module a un nom et rassemble des définitions et il peut être une extension d'autres modules.
 - $[f \text{ EXCEPT! } [i] = e]$ est la fonction f où seule la valeur en i a changé et vaut .
- ▶ Une configuration doit être définie pour évaluer une spécification

- ▶ Limitation des actions :

$$\begin{aligned} \text{nom} &\triangleq \\ &\wedge \text{cond}(v, w) \\ &\wedge v' = e(v, w) \\ &\wedge w' = w \end{aligned}$$

- ▶ $e(v, w)$ doit être codable en Java.
- ▶ Modules standards : Naturals, Integers, TLC ...

- ▶ Téléchargez l'application le site de Microsoft pour votre ordinateur.
- ▶ Ecrivez des modèles et testez les !
- ▶ Limitations par les domaines des variables.



Permettre un raisonnement symbolique quel que soit l'ensemble des états

- 1 Identify the input data and the results produced : domains, types, conditions.
- 2 Normalise the code in the form of a contract defining the precondition and the postcondition : formalisation of the code in the form of a function, modified variables, unmodified variables, the *requires* assertion, and the *ensures* relation.
- 3 Annotation of the code of the procedure : control points, assertions for each point.
- 4 Generating proof obligations for partial correctness : PO(PC).
- 5 Checking proof obligations for partial correctness.